

Proyecto Realizado por Carlos Díaz

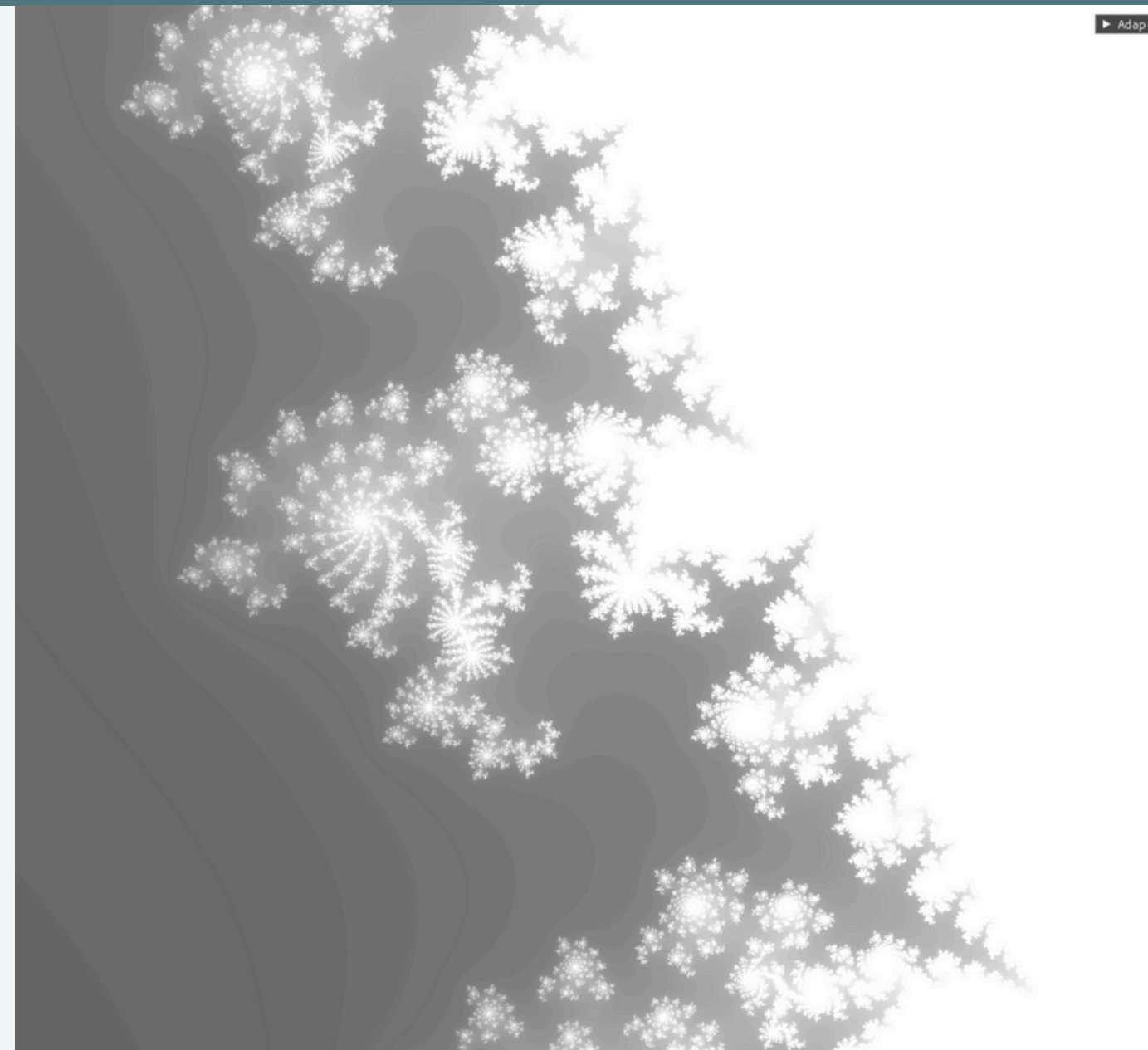
VISUALIZACIÓN DE FRACTALES



¿QUÉ SON LOS FRACTALES Y POR QUÉ VISUALIZARLOS?

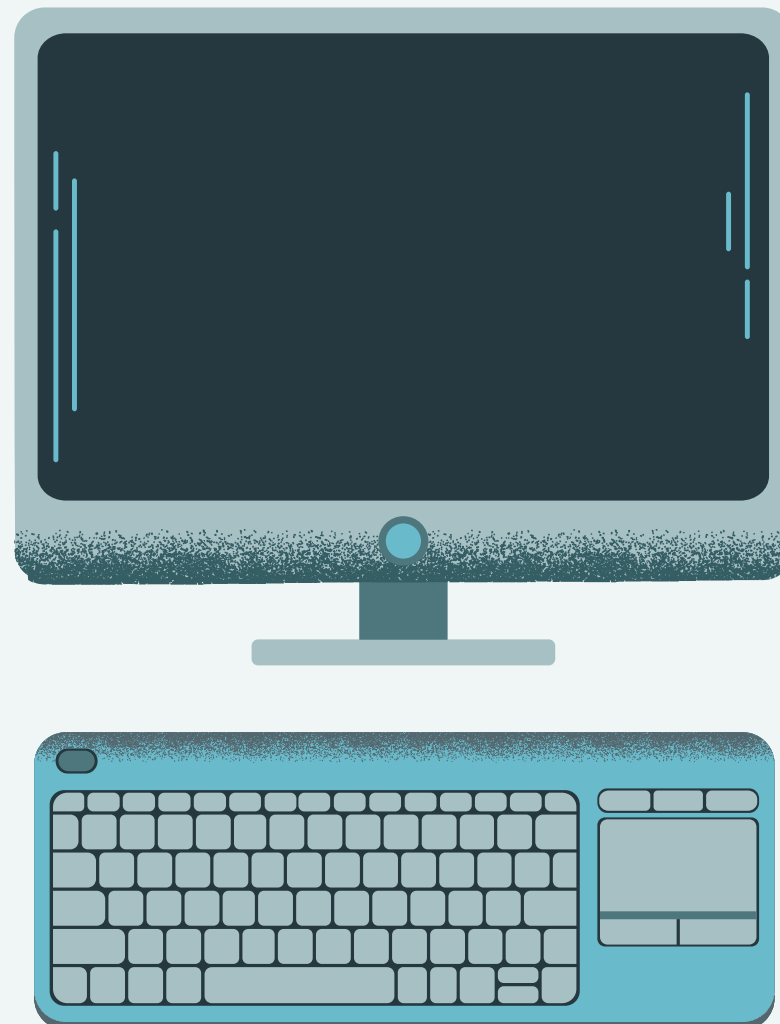
Los fractales son objetos matemáticos cuya característica principal es la auto-similitud: sus partes se asemejan al todo, sin importar cuánto nos acerquemos. Esto les confiere una complejidad y detalle visualmente infinitos.

Este proyecto se motivó por la belleza de estas estructuras y el desafío técnico que implica renderizarlas, especialmente en 3D, donde no tenemos mallas poligonales tradicionales



ESTRUCTURA DEL VISUALIZADOR

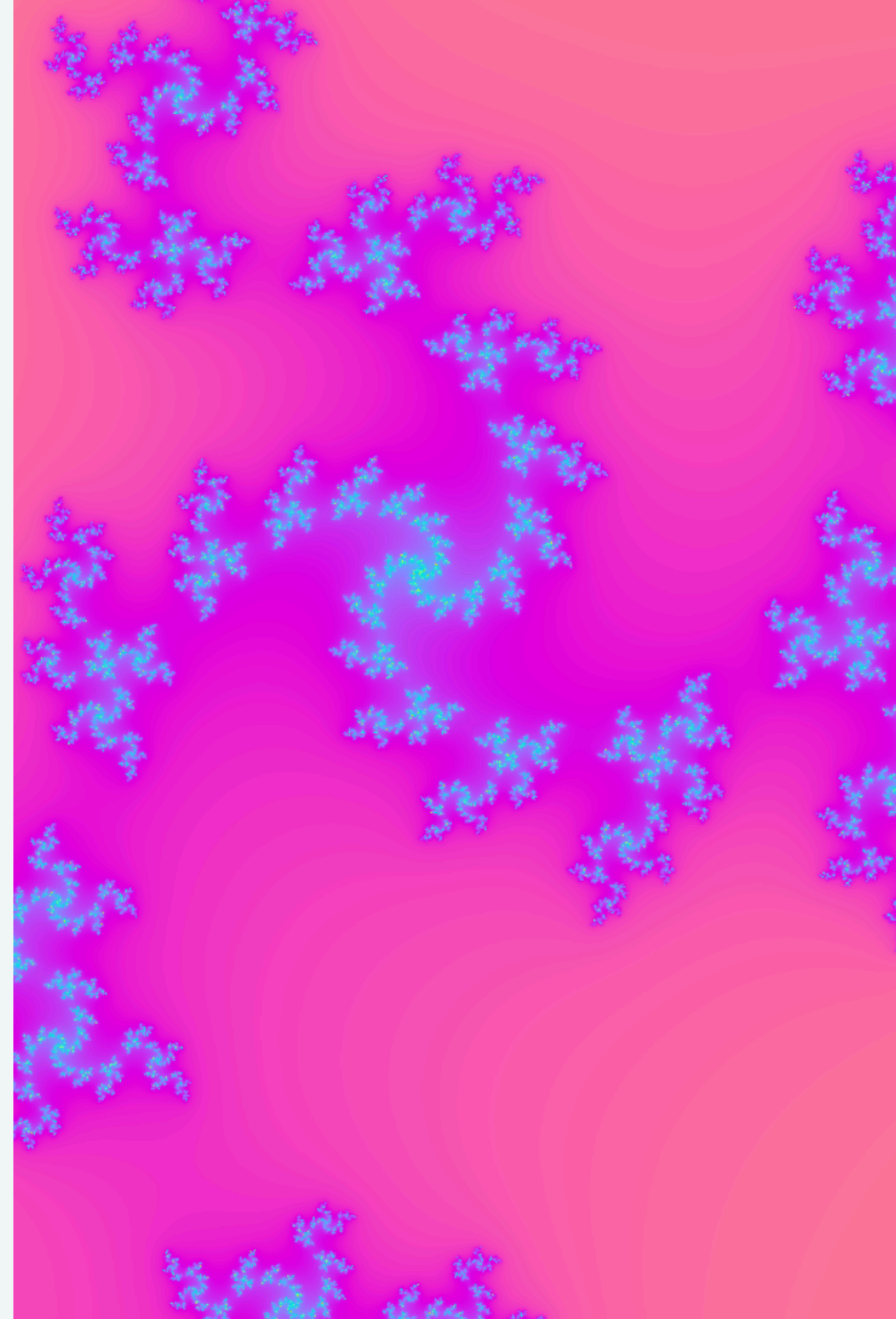
- El visualizador se construyó en C++, utilizando Diligent Engine principalmente como una API gráfica para interactuar con la GPU mediante HLSL
- La interfaz de usuario, desarrollada con ImGui, permite controlar en tiempo real los parámetros del fractal, la cámara y las opciones de renderizado

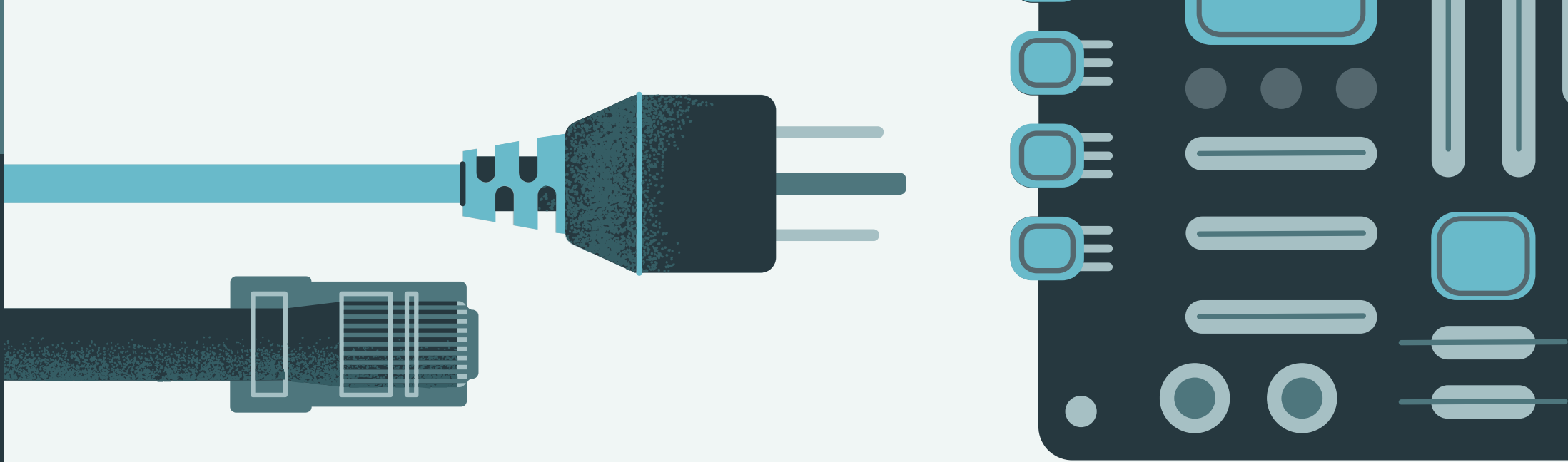
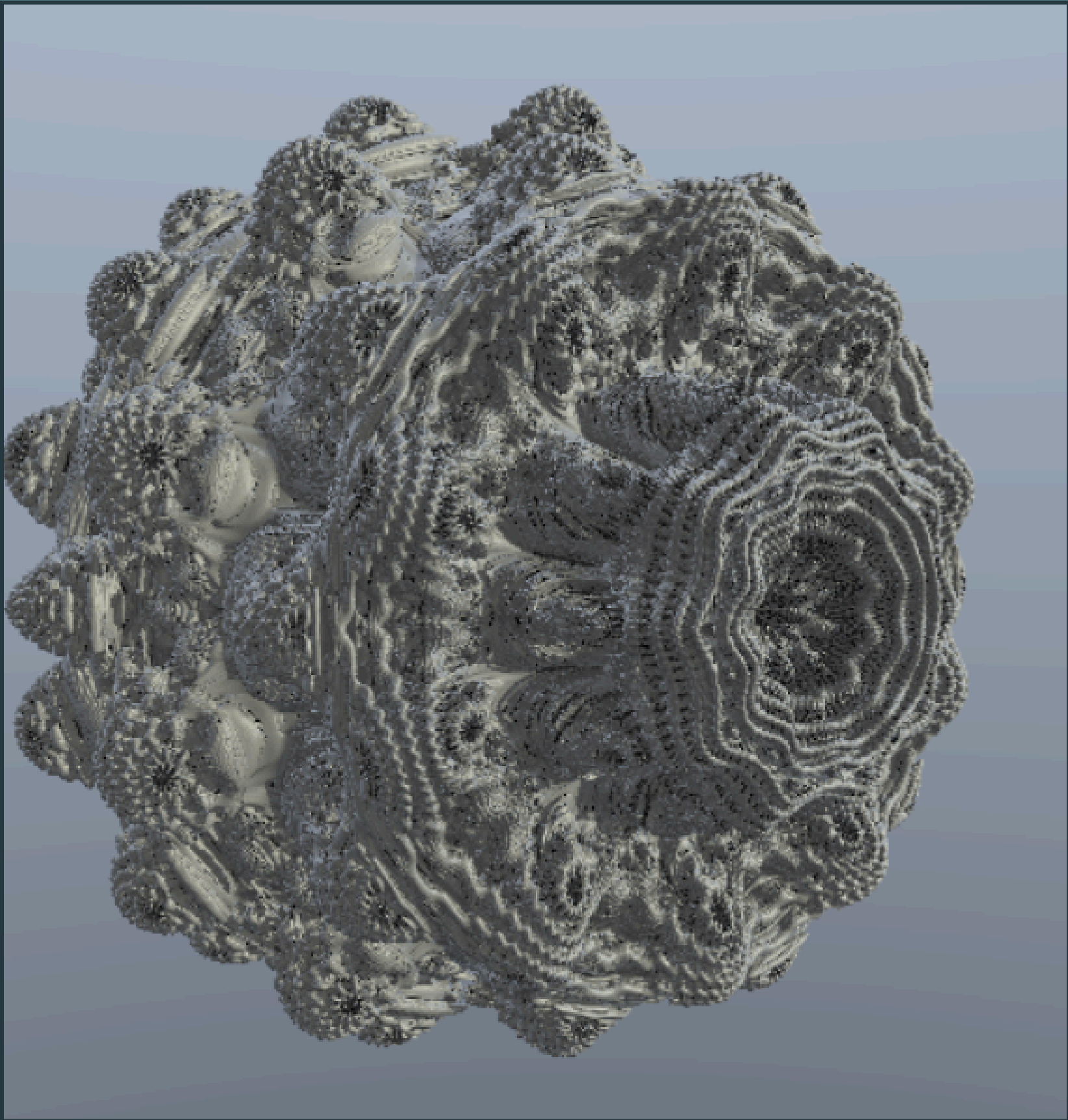


- La lógica principal en C++ gestiona estos parámetros y los envía al shader de la GPU, donde ocurre la magia del cálculo y renderizado del fractal
- Implementamos dos pipelines de renderizado: uno tradicional basado en pixel shaders para fractales 2D y un pipeline de compute shader para los fractales 3D más demandantes, lo que nos permite aprovechar mejor el paralelismo de la GPU

EXPLORACIÓN DE FRACTALES 2D

- Fractales implementados: Mandelbrot, Burning Ship, Phoenix/Julia.
- El color de cada píxel se determina por cuántas iteraciones tarda el punto en 'escapar' de una región definida, o si no escapa después de un máximo de iteraciones
- Implementamos controles para el zoom, el desplazamiento, el número de iteraciones y, crucialmente, una opción de cálculo en doble precisión para explorar los detalles increíblemente finos que emergen al hacer zoom profundo, evitando artefactos de precisión

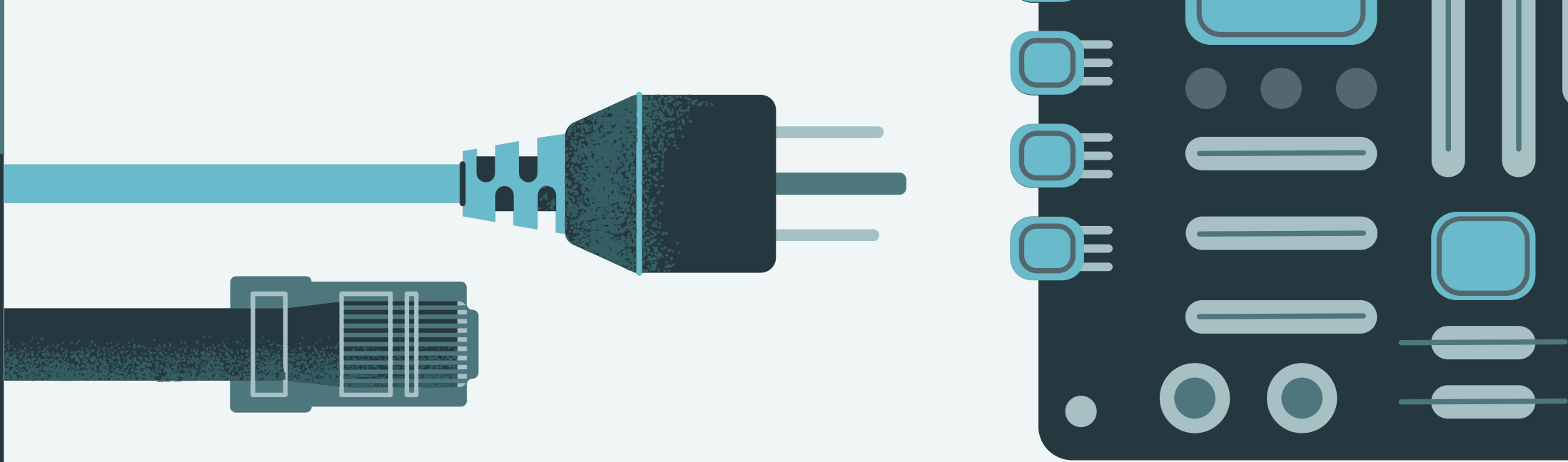
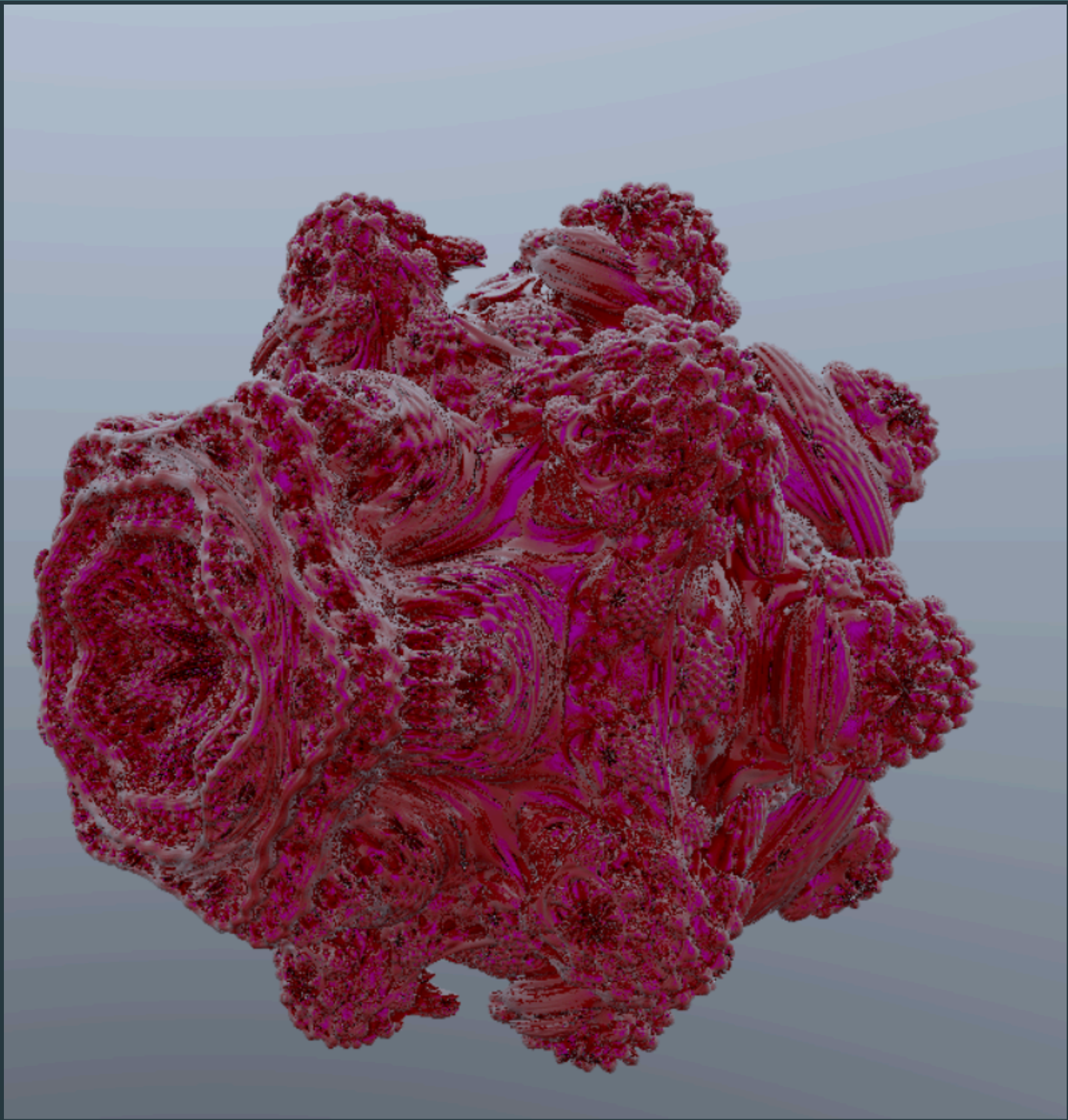




RAYMARCHING

Para los fractales 3D, como el Mandelbulb o el Menger Sponge, no podemos usar la rasterización tradicional de polígonos. Estos objetos se definen implícitamente por fórmulas matemáticas

La técnica principal que usamos es el Raymarching. Por cada píxel, lanzamos un rayo virtual hacia la escena



RAYMARCHING

Luego, 'marchamos' a lo largo de este rayo en pasos. La clave aquí es la Función de Distancia Estimada, o SDF.

Esta función nos dice, para cualquier punto en el espacio, cuán lejos estamos de la superficie del fractal

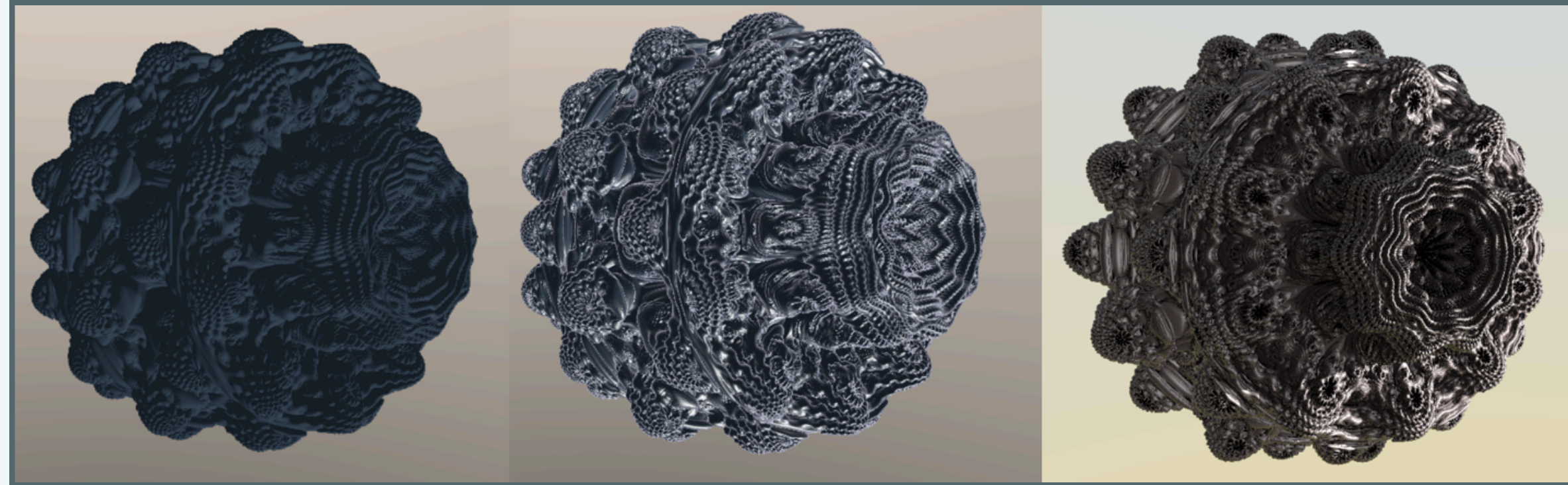
Si la distancia es muy pequeña, hemos encontrado la superficie. Si no, avanzamos el rayo esa distancia y repetimos. Esto nos permite renderizar geometrías infinitamente complejas sin necesidad de polígonos

MEJORANDO EL REALISMO

- PATH TRACING PARA FRACTALES 3D

Aunque el raymarching nos permite ver la forma del fractal, para un realismo mayor implementamos Path Tracing, específicamente para el Mandelbulb y luego para el Menger Sponge

El Path Tracing simula cómo la luz interactúa realmente con las superficies. En lugar de detener el rayo en el primer impacto, permitimos que rebote múltiples veces en la escena



VISUALIZANDO EL MENGINEER SPONGE (LABERINTO INFINITO)

Un desafío particular fue la visualización del Menger Sponge con path tracing. Inicialmente, nos encontramos con artefactos visuales, como patrones extraños o la pérdida de la estructura de laberinto infinito

Teóricamente, estos problemas surgían de la Función de Distancia Estimada. Si la SDF no devuelve la distancia correcta, o si su signo es inconsistente con lo que espera el algoritmo de raymarching, el rayo puede tomar pasos incorrectos, converger en superficies falsas o quedarse atascado

